

Software Tools and Techniques for CSE: Assessment 4

Project Report

Ridham Patel (23110238), Shardul Junagade (23110297)

20 November 2025

Computer Science & Engineering
Indian Institute of Technology Gandhinagar
India

Repository Link: `cs202-stt`

Contents

1	Lab 11: Events and Delegates in C#	2
1.1	Introduction	2
1.2	Tools	2
1.3	Setup	2
1.4	Methodology and Execution	4
1.4.1	Part 1 - Basic Custom Events (Color & Text)	4
1.4.2	Part 2 - EventArgs + Multicast	7
1.4.3	Task 3 - Output Reasoning (Level 0)	11
1.4.4	Task 4 - Output Reasoning (Level 1)	13
1.4.5	Task 5 - Output Reasoning (Level 2)	15
1.5	Conclusion	16
1.6	References	17

1 Lab 11: Events and Delegates in C#

Metadata

- Course: CS202 - Software Tools and Techniques for CSE
- Lab Topic: Events and Delegates in C# Windows Forms Applications
- Name: Shardul Junagade (23110297)
- Date: 3rd November 2025

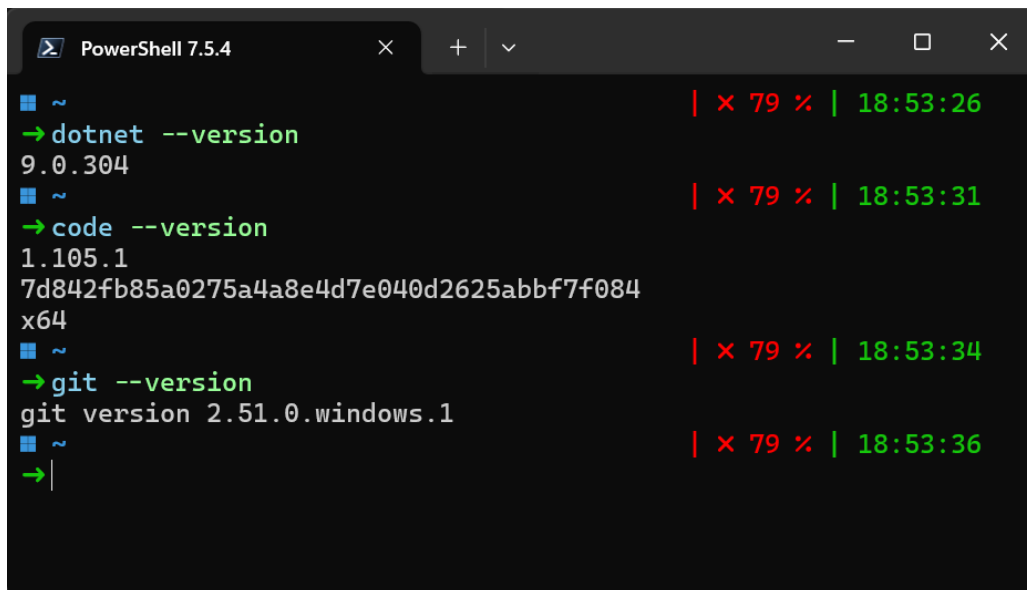
Repository path: lab11/

1.1 Introduction

I completed Laboratory Session 11 on events and delegates in C# Windows Forms. The session centered on building a small interactive form and then reasoning through several delegate/event code snippets to understand chaining, thresholds, and nested invocation. In this lab I focused on wiring custom events between UI controls, extending a basic event with a custom `EventArgs` payload, and demonstrating multicast behavior. After that I manually traced five embedded C# code snippets (Tasks 3–5) and wrote out their outputs with reasoning. I kept the implementation compact and readable rather than over-engineered.

1.2 Tools

- OS: Windows 11; Shell: PowerShell 7
- .NET SDK: .NET 8 (meets the “.NET 6 or later” requirement)
- IDE/Editor: Visual Studio 2022 Community
- Git: for version control



```
PowerShell 7.5.4
~ | x 79 % | 18:53:26
→ dotnet --version
9.0.304
~ | x 79 % | 18:53:31
→ code --version
1.105.1
7d842fb85a0275a4a8e4d7e040d2625abbf7f084
x64
~ | x 79 % | 18:53:34
→ git --version
git version 2.51.0.windows.1
~ | x 79 % | 18:53:36
→
```

Figure 1: Environment Details

1.3 Setup

I reused the same .NET setup as in earlier labs and verified it before working on the Windows Forms app.

Visual Studio path

1. Install Visual Studio 2022 (Community Edition) with the “.NET desktop development” workload selected.

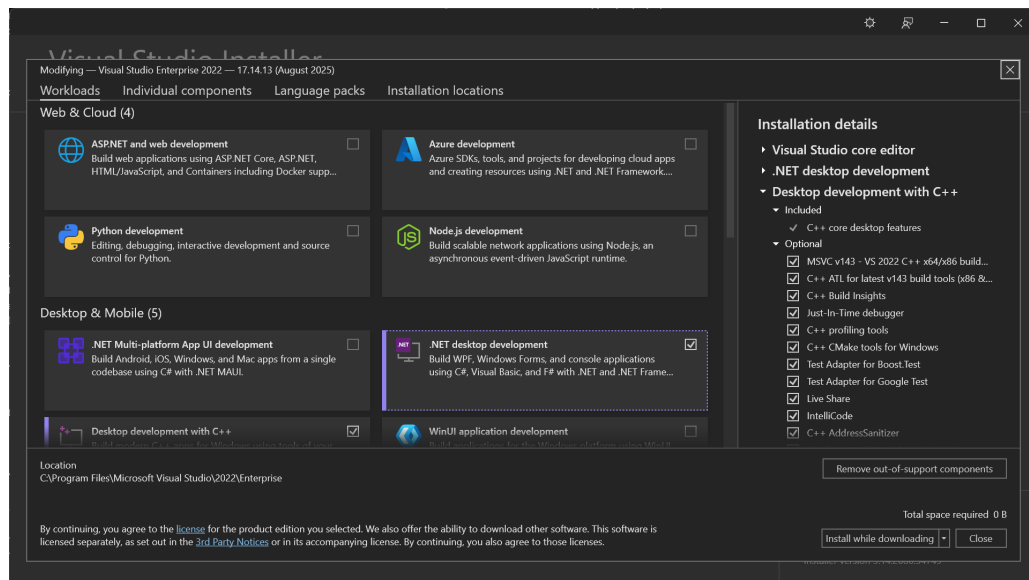


Figure 2: Workload Selection

2. Confirm the SDK is on PATH:

```
dotnet --version
```

3. Create a new C# Windows Forms App in Visual Studio and set the Target Framework to .NET 8.0 in project properties.

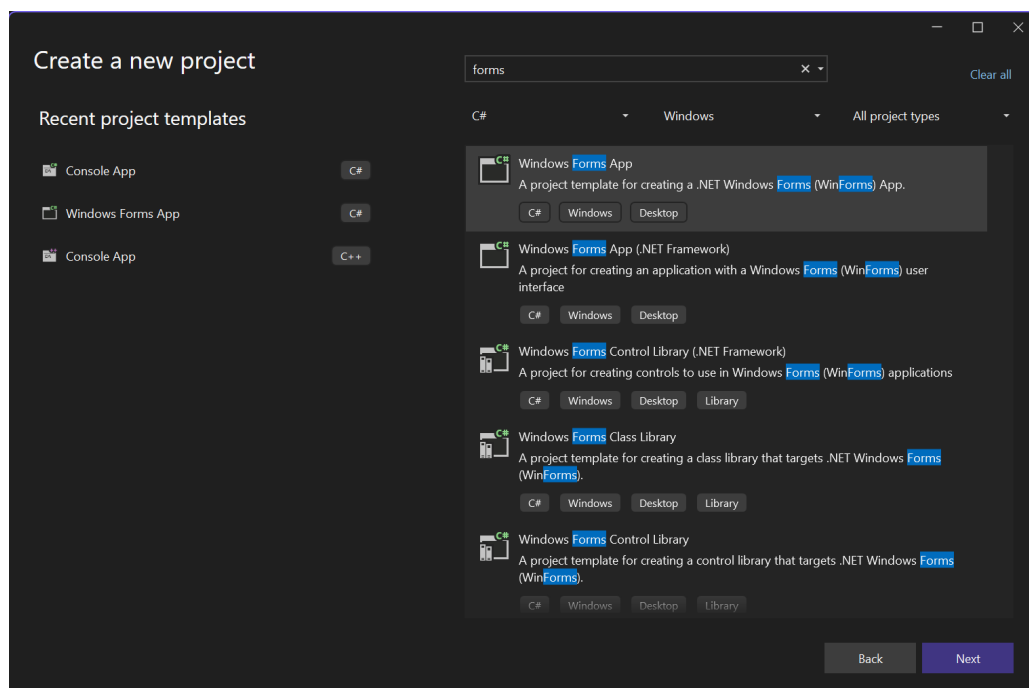


Figure 3: Project Target Framework

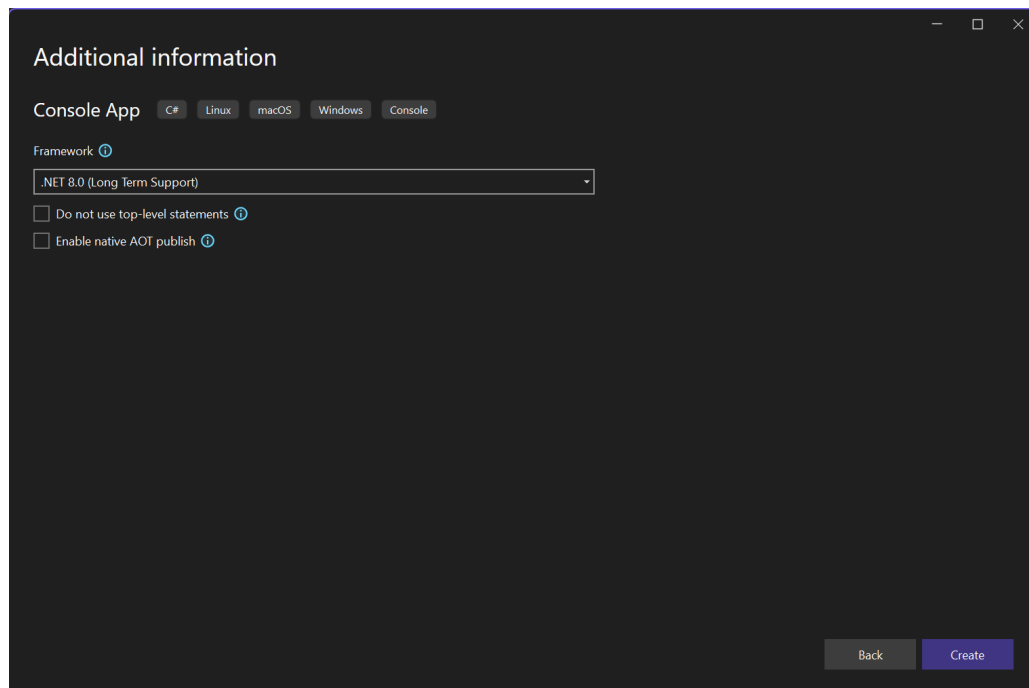


Figure 4: .NET Version Selection

1.4 Methodology and Execution

I then designed the form with two buttons (`btnChangeColor`, `btnChangeText`), one label (`lblMessage` initialised to “Welcome to Events Lab”), and a ComboBox (`cmbColors`) where I added the entries Red, Green, Blue via the Properties window.

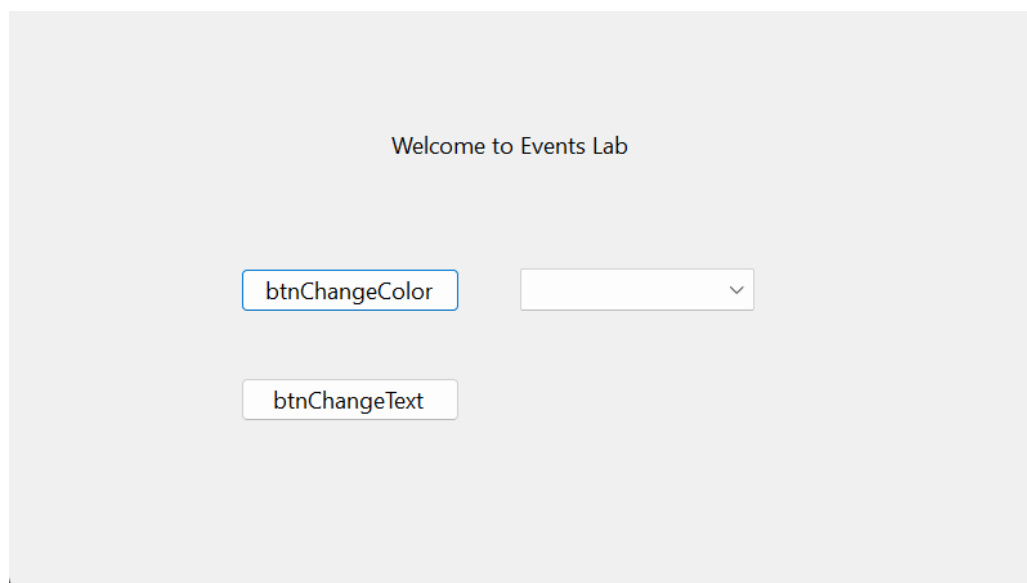


Figure 5: Form Design

1.4.1 Part 1 - Basic Custom Events (Color & Text)

In `Form1.cs` I declared two delegates and the corresponding events:

```
public delegate void ColorChangedHandler(System.Drawing.Color newColor);
public delegate void TextChangedHandler(string newText);
public event ColorChangedHandler ColorChangedEvent;
public event TextChangedHandler TextChangedEvent;
```

In the form constructor I subscribed the handlers:

```
ColorChangedEvent += UpdateLabelColor;
TextChangedEvent += UpdateLabelText;
```

On the colour button click I read the selected ComboBox string, mapped it to a `System.Drawing.Color` with a `switch`, and invoked the custom event:

```
ColorChangedEvent?.Invoke(selectedColor); // raises custom event
```

The text button built a timestamp string and invoked:

```
TextChangedEvent?.Invoke(newText);
```

The label foreground and caption updated only through these events - the click handlers acted as publishers, and the subscriber methods performed the UI changes.

```
1 namespace EventPlayground
2 {
3     public partial class Form1 : Form
4     {
5         // Custom delegates
6         public delegate void ColorChangedHandler(System.Drawing.Color newColor);
7         public delegate void TextChangedHandler(string newText);
8
9         // Events based on those delegates
10        public event ColorChangedHandler ColorChangedEvent;
11        public event TextChangedHandler TextChangedEvent;
12
13        // This will be called when ColorChangedEvent is raised
14        private void UpdateLabelColor(System.Drawing.Color newColor)
15        {
16            lblMessage.ForeColor = newColor;
17        }
18
19        // This will be called when TextChangedEvent is raised
20        private void UpdateLabelText(string newText)
21        {
22            lblMessage.Text = newText;
23        }
24
25        public Form1()
26        {
27            InitializeComponent();
28            // Subscribe methods to our custom events
29            ColorChangedEvent += UpdateLabelColor;
30            TextChangedEvent += UpdateLabelText;
31        }
32
33        private void Form1_Load(object sender, EventArgs e)
34        {
35
36        }
37    }
38 }
```

Figure 6: Part 1 - Code Snippet of Form1.cs

```
38 private void btnChangeColor_Click(object sender, EventArgs e)
39 {
40     // Get selected colour name from ComboBox
41     string? selectedColorName = cmbColors.SelectedItem?.ToString();
42
43     if (string.IsNullOrEmpty(selectedColorName))
44     {
45         MessageBox.Show("Please select a colour from the list.");
46         return;
47     }
48
49     // Convert name to actual Color
50     System.Drawing.Color selectedColor;
51
52     switch (selectedColorName)
53     {
54         case "Red":
55             selectedColor = System.Drawing.Color.Red;
56             break;
57         case "Green":
58             selectedColor = System.Drawing.Color.Green;
59             break;
60         case "Blue":
61             selectedColor = System.Drawing.Color.Blue;
62             break;
63         default:
64             // fallback
65             selectedColor = System.Drawing.Color.Black;
66             break;
67     }
68
69     // Raise the custom event (if there are subscribers)
70     ColorChangedEvent?.Invoke(selectedColor);
71 }
72
73 private void btnChangeText_Click(object sender, EventArgs e)
74 {
75     // build a new text with current date & time
76     string newText = "Current date and time: " + DateTime.Now.ToString("dd/MM/yyyy HH:mm:ss");
77
78     // raise custom event
79     TextChangedEvent?.Invoke(newText);
80 }
81
82 private void cmbColors_SelectedIndexChanged(object sender, EventArgs e)
83 {
84 }
85
86 }
87
88 }
```

Figure 7: Part 1 - Code Snippet of Form1.cs

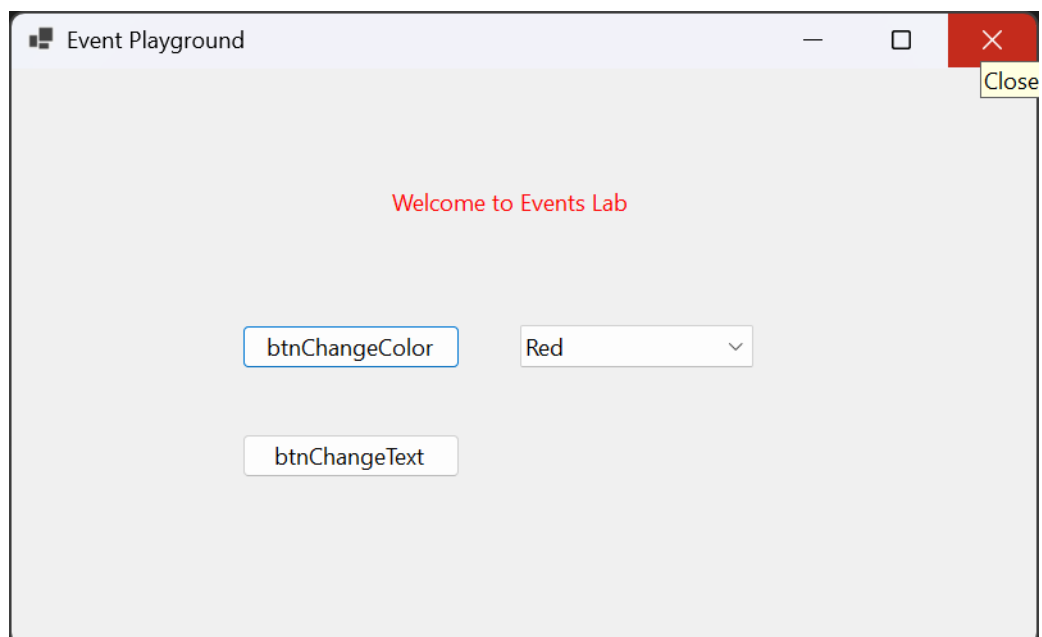


Figure 8: Part 1 - GUI after colour change

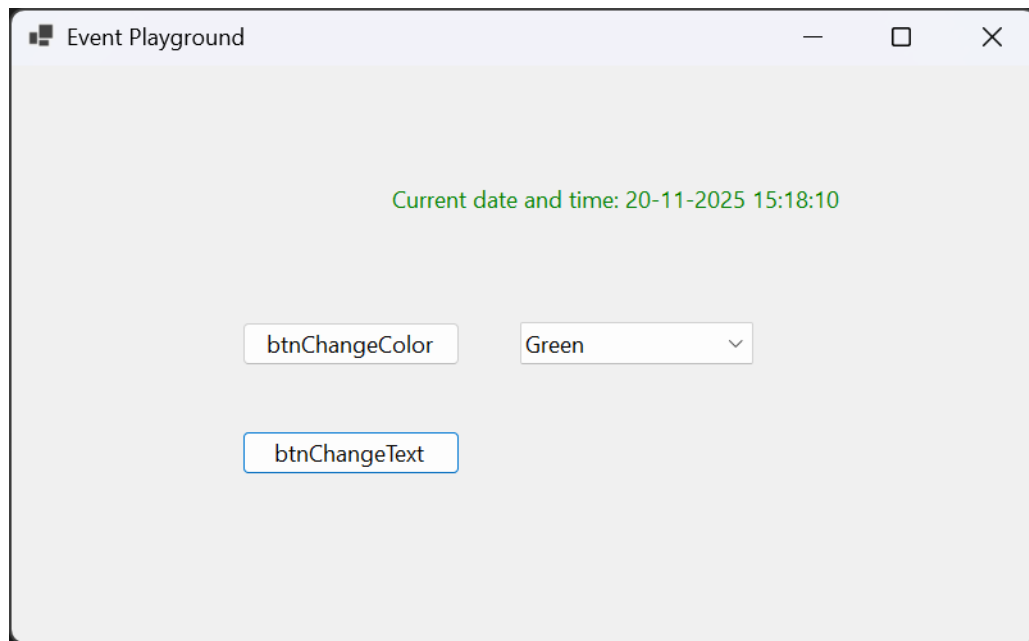


Figure 9: Part 1 - GUI after text change

1.4.2 Part 2 - EventArgs + Multicast

For the second part I extended the colour change event to carry structured information. I added a `ColorEventArgs` class:

```
public class ColorEventArgs : EventArgs
{
    public string ColorName { get; }

    public ColorEventArgs(string colorName)
    {
        ColorName = colorName;
    }
}
```

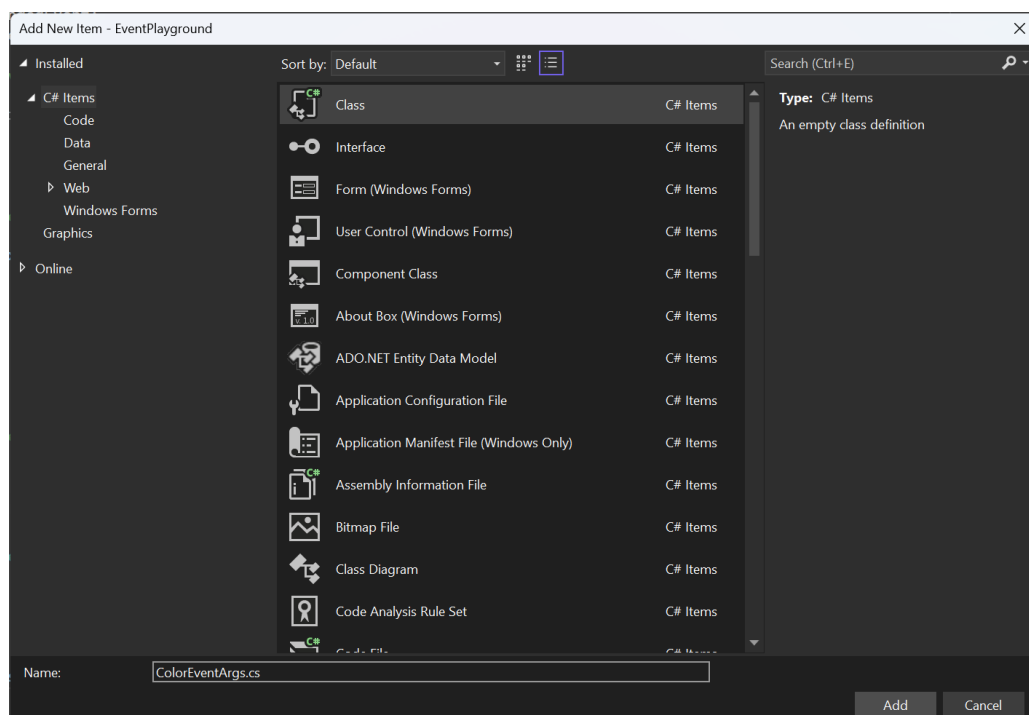


Figure 10: ColorEventArgs class

Then in Form1.cs I adopted the conventional sender + args pattern for the event:

```
public delegate void ColorChangedHandler(object sender, ColorEventArgs e);  
public event ColorChangedHandler ColorChangedEvent;
```

In the constructor I wired two subscribers to demonstrate multicast behaviour:

```
ColorChangedEvent += UpdateLabelColor;  
ColorChangedEvent += ShowNotification; // second subscriber
```

On the button click I created the EventArgs and invoked the event:

```
var args = new ColorEventArgs(selectedColorName);  
ColorChangedEvent?.Invoke(this, args);
```

UpdateLabelColor mapped the ColorName string to a System.Drawing.Color, and ShowNotification popped up a message box confirming the choice. Both ran in order when I clicked the button, this showed multiple subscribers on the same event.

```
1  using System;  
2  using System.Collections.Generic;  
3  using System.Linq;  
4  using System.Text;  
5  using System.Threading.Tasks;  
6  
7  namespace EventPlayground  
8  {  
9      public class ColorEventArgs : EventArgs  
10     {  
11         public string ColorName { get; }  
12  
13         public ColorEventArgs(string colorName)  
14         {  
15             ColorName = colorName;  
16         }  
17     }  
18 }  
19
```

Figure 11: ColorEventArgs code snippet


```
1  using System;
2  using System.Windows.Forms;
3
4  namespace EventPlayground
5  {
6      public partial class Form1 : Form
7      {
8          // Custom delegates
9          public delegate void ColorChangedHandler(object sender, ColorEventArgs e);
10         public delegate void TextChangedHandler(string newText);
11
12         // Events based on those delegates
13         public event ColorChangedHandler ColorChangedEvent;
14         public event TextChangedHandler TextChangedEvent;
15
16         // This will be called when ColorChangedEvent is raised
17         private void UpdateLabelColor(object sender, ColorEventArgs e)
18         {
19             // Convert color name from EventArgs to actual Color
20             System.Drawing.Color newColor;
21
22             switch (e.ColorName)
23             {
24                 case "Red":
25                     newColor = System.Drawing.Color.Red;
26                     break;
27                 case "Green":
28                     newColor = System.Drawing.Color.Green;
29                     break;
30                 case "Blue":
31                     newColor = System.Drawing.Color.Blue;
32                     break;
33                 default:
34                     newColor = System.Drawing.Color.Black;
35                     break;
36             }
37
38             lblMessage.ForeColor = newColor;
39         }
40
41         // This will be called when TextChangedEvent is raised
42         private void UpdateLabelText(string newText)
43         {
44             lblMessage.Text = newText;
45         }
46
47         private void ShowNotification(object sender, ColorEventArgs e)
48         {
49             MessageBox.Show($"Selected colour: {e.ColorName}", "Colour Changed");
50         }
51     }
```

Figure 12: Part 2 - Code Snippet of Form1.cs

```

52     public Form1()
53     {
54         InitializeComponent();
55
56         // ColorChangedEvent now has 2 subscribers (multicast)
57         ColorChangedEvent += UpdateLabelColor;
58         ColorChangedEvent += ShowNotification;
59
60         // TextChangedEvent still has one
61         TextChangedEvent += UpdateLabelText;
62     }
63
64     private void Form1_Load(object sender, EventArgs e)
65     {
66     }
67
68
69     private void btnChangeColor_Click(object sender, EventArgs e)
70     {
71         string? selectedColorName = cmbColors.SelectedItem?.ToString();
72
73         if (string.IsNullOrEmpty(selectedColorName))
74         {
75             MessageBox.Show("Please select a colour from the list.");
76             return;
77         }
78
79         // Create EventArgs with the selected colour name
80         ColorEventArgs args = new ColorEventArgs(selectedColorName);
81
82         // Raise the custom event with (sender, args)
83         ColorChangedEvent?.Invoke(this, args);
84     }
85
86     private void btnChangeText_Click(object sender, EventArgs e)
87     {
88         // build a new text with current date & time
89         string newText = "Current date and time: " + DateTime.Now.ToString("dd/MM/yyyy HH:mm:ss");
90
91         // raise custom event
92         TextChangedEvent?.Invoke(newText);
93     }
94
95     private void cmbColors_SelectedIndexChanged(object sender, EventArgs e)
96     {
97     }
98
99
100 }
101

```

Figure 13: Part 2 - Code Snippet of Form1.cs

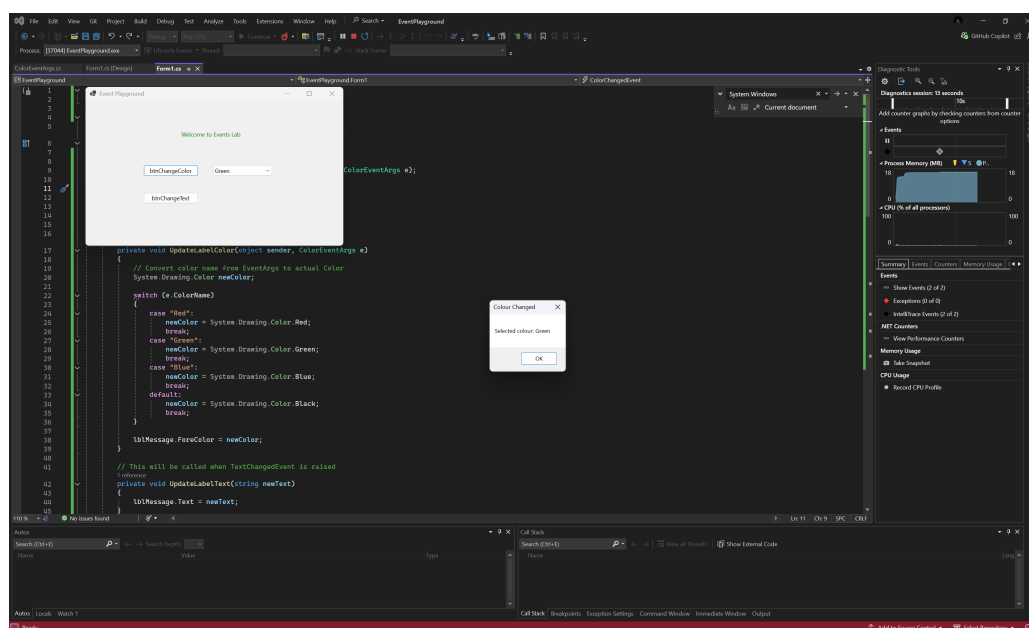
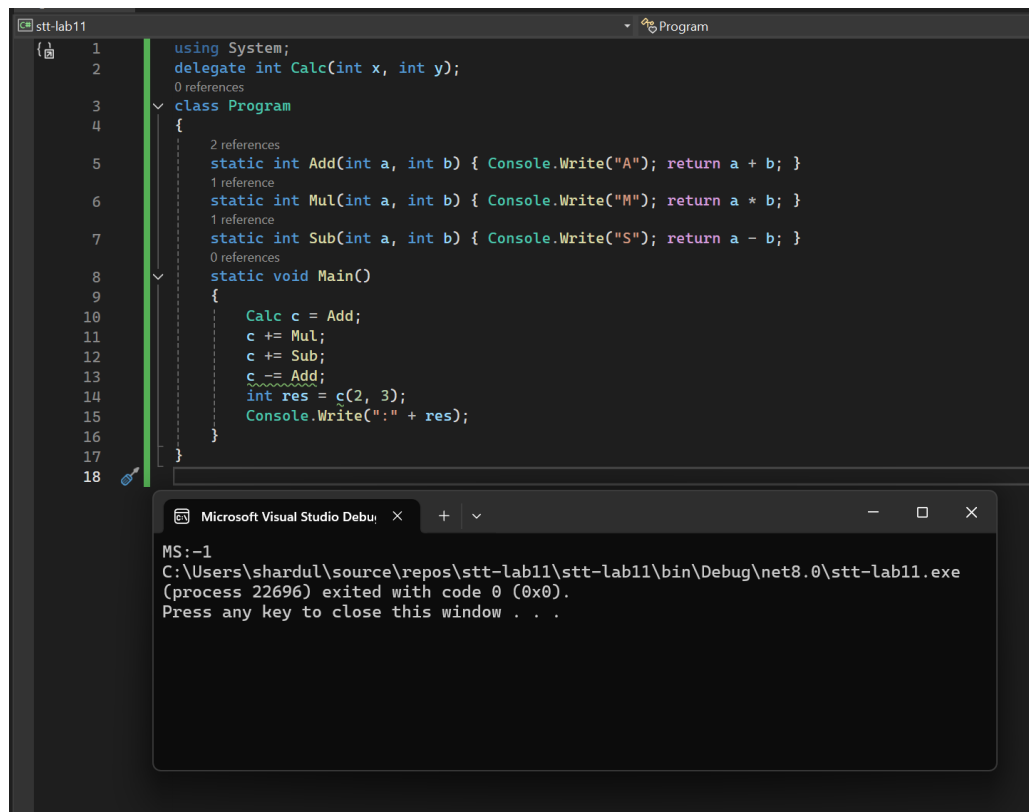


Figure 14: Part 2 - GUI and message box after colour change

1.4.3 Task 3 - Output Reasoning (Level 0)

Q1. Final output:

MS:-1



The screenshot displays a Visual Studio IDE with a C# program named 'stt-lab11'. The code defines a delegate 'Calc' and a class 'Program' with three static methods: 'Add', 'Mul', and 'Sub'. The 'Main' method initializes a 'Calc' delegate 'c' with 'Add', then adds 'Mul' and 'Sub' to it. Finally, it calls 'c(2, 3)' and prints the result. The output window shows the execution results: 'MS:-1', the full path to the executable, and the exit code '0 (0x0)'.

```
1 using System;
2 delegate int Calc(int x, int y);
3
4 class Program
5 {
6     static int Add(int a, int b) { Console.Write("A"); return a + b; }
7     static int Mul(int a, int b) { Console.Write("M"); return a * b; }
8     static int Sub(int a, int b) { Console.Write("S"); return a - b; }
9
10    static void Main()
11    {
12        Calc c = Add;
13        c += Mul;
14        c += Sub;
15        c -= Add;
16        int res = c(2, 3);
17        Console.WriteLine(":" + res);
18    }
19 }
```

Microsoft Visual Studio Debug Console Output:

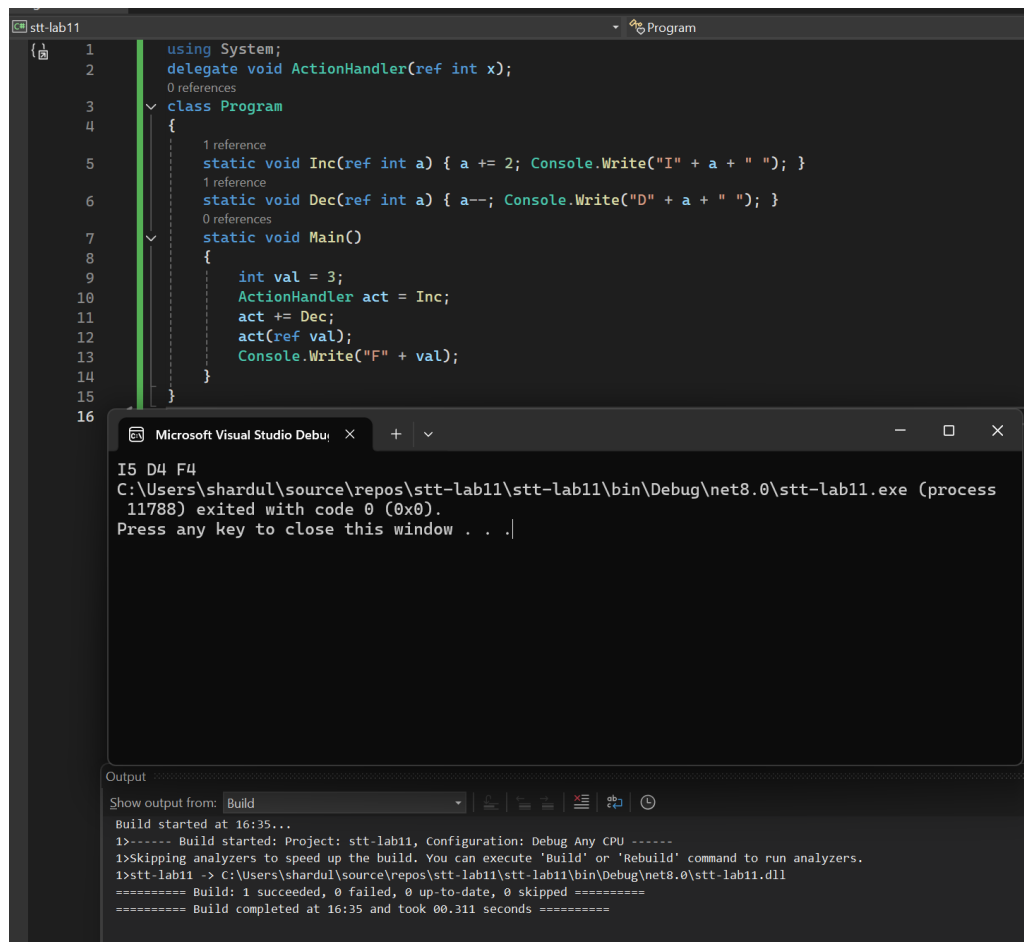
```
MS:-1
C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.exe
(process 22696) exited with code 0 (0x0).
Press any key to close this window . . .
```

Figure 15: Task 3 Q1 - code and output

Reasoning: the delegate started with Add, then had Mul and Sub added, and finally Add was removed. That left the invocation list as [Mul, Sub]. Calling Mul(2,3) printed M and returned 6; calling Sub(2,3) printed S and returned -1. For multicast delegates with a return type, only the value from the last method is used, so the printed result became :-1.

Q2. Final output:

I5 D4 F4



The screenshot shows a Visual Studio IDE with a C# project named 'stt-lab11'. The code defines a delegate `ActionHandler` and a class `Program` with two static methods, `Inc` and `Dec`, which modify a value and print it. The `Main` method initializes `val` to 3, adds `Inc` to a multicast list, then `Dec`, and finally calls `act(ref val)` and prints the result.

```
1 using System;
2 delegate void ActionHandler(ref int x);
3
4 class Program
5 {
6     static void Inc(ref int a) { a += 2; Console.WriteLine("I" + a + " "); }
7     static void Dec(ref int a) { a--; Console.WriteLine("D" + a + " "); }
8
9     static void Main()
10    {
11        int val = 3;
12        ActionHandler act = Inc;
13        act += Dec;
14        act(ref val);
15        Console.WriteLine("F" + val);
16    }
17 }
```

The output window shows the execution results:

```
I5 D4 F4
C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.exe (process 11788) exited with code 0 (0x0).
Press any key to close this window . . .|
```

The bottom pane shows the build output:

```
Output
Show output from: Build
Build started at 16:35...
1>----- Build started: Project: stt-lab11, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab11 -> C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 16:35 and took 00.311 seconds =====
```

Figure 16: Task 3 Q2 - code and output

Reasoning: with `val = 3`, the invocation list was `[Inc, Dec]`. `Inc(ref val)` increased it to 5 and printed `I5` . Then `Dec(ref val)` dropped it to 4 and printed `D4` . After the multicast call the variable held 4, so the final print produced `F4`.

1.4.4 Task 4 - Output Reasoning (Level 1)

Q1. Final output:

>1>2[M2]>3[L3](Reset)>4[M4{Alert}]>5>6[M6][L6](Reset)

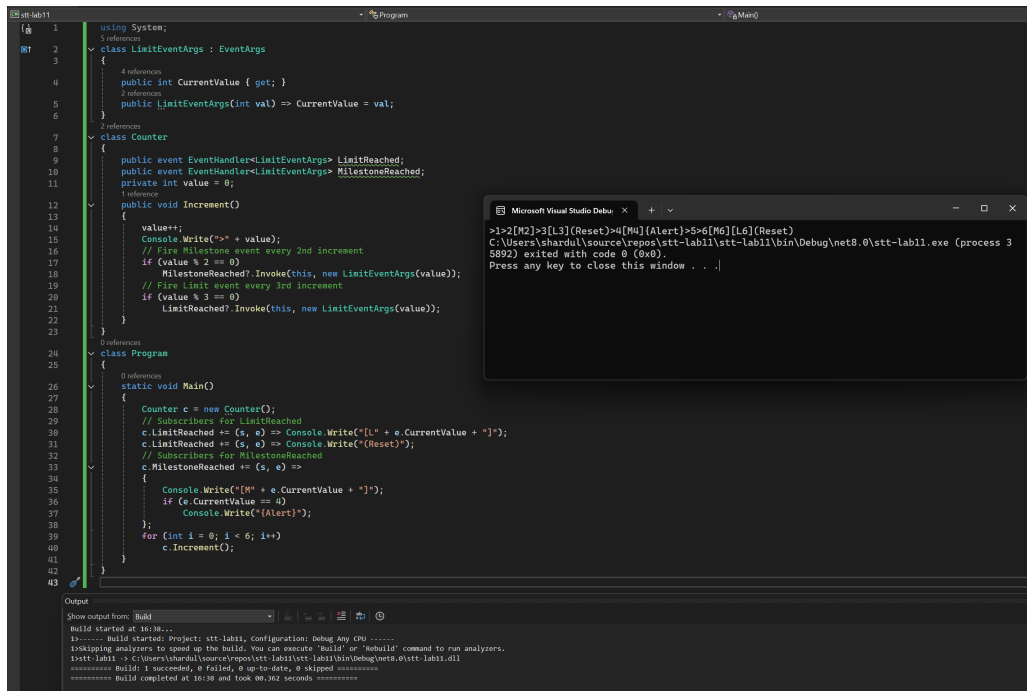


Figure 17: Task 4 Q1 - code and output

Reasoning: the `Increment()` method first increases the internal counter and immediately prints the current value in the form `>value`. After that, two conditional checks determine whether additional text is printed.

If the value is even (`value % 2 == 0`), the `MilestoneReached` event is raised, and its subscriber prints `[Mvalue]`. For the special case where the value is 4, the same subscriber also appends `{Alert}`. If the value is a multiple of 3 (`value % 3 == 0`), the `LimitReached` event is raised, and two subscribers print `[Lvalue]` and `(Reset)` in that order.

The loop in `Main` calls `Increment()` six times, so the counter takes the values 1 through 6, and the printed output for each call is as follows:

- Value 1: only `>1` (no event conditions satisfied).
- Value 2: `>2` and `[M2]` (even, milestone only).
- Value 3: `>3[L3](Reset)` (multiple of 3, limit reached only).
- Value 4: `>4[M4{Alert}]` (even, milestone with alert; not a multiple of 3).
- Value 5: only `>5` (no event conditions satisfied).
- Value 6: `>6[M6][L6](Reset)` (both even and multiple of 3, so milestone and limit events fire).

Concatenating these pieces in order produces the final output:

>1>2[M2]>3[L3](Reset)>4[M4{Alert}]>5>6[M6][L6](Reset)

Q2. Final output:

```

Temperature changed from 30 C to 46 C
Warning: Sudden change detected!
Temperature changed from 46 C to 52 C

```

The screenshot shows a Visual Studio IDE with two files open: `stt-lab11` and `Program`. The `TemperatureEventArgs` class is defined with `OldTemperature` and `NewTemperature` properties and a constructor. The `TemperatureSensor` class implements an `EventHandler<TemperatureEventArgs>` interface with a `TemperatureChanged` event and an `UpdateTemperature` method that triggers the event if the change is greater than 5 degrees. The `Program` class contains a `Main` method that creates a `TemperatureSensor` instance and updates its temperature from 25 to 28, 30, 46, and 52. The console window shows the output: "Temperature changed from 30°C to 46°C", "Warning: Sudden change detected!", and "Temperature changed from 46°C to 52°C". The build output at the bottom shows the project was built successfully.

```

using System;
3 references
class TemperatureEventArgs : EventArgs
{
    3 references
    public int OldTemperature { get; }
    3 references
    public int NewTemperature { get; }
    1 reference
    public TemperatureEventArgs(int oldTemp, int newTemp)
    {
        OldTemperature = oldTemp;
        NewTemperature = newTemp;
    }
}
2 references
class TemperatureSensor
{
    public event EventHandler<TemperatureEventArgs> TemperatureChanged;
    private int temperature = 25;
    4 references
    public void UpdateTemperature(int newTemp)
    {
        int oldTemp = temperature;
        temperature = newTemp;
        if (Math.Abs(newTemp - oldTemp) > 5)
        {
            TemperatureChanged?.Invoke(this, new TemperatureEventArgs(oldTemp, newTemp));
        }
    }
}
0 references
class Program
{
    0 references
    static void Main()
    {
        TemperatureSensor sensor = new TemperatureSensor();
        sensor.TemperatureChanged += (s, e) =>
        {
            Console.WriteLine($"Temperature changed from {e.OldTemperature}°C to {e.NewTemperature}°C");
            sensor.TemperatureChanged += (s, e) =>
            {
                if (Math.Abs(e.NewTemperature - e.OldTemperature) > 10)
                {
                    Console.WriteLine("Warning: Sudden change detected!");
                }
            };
        };
        sensor.UpdateTemperature(28);
        sensor.UpdateTemperature(30);
        sensor.UpdateTemperature(46);
        sensor.UpdateTemperature(52);
    }
}

```

```

Microsoft Visual Studio Debu
Temperature changed from 30°C to 46°C
Warning: Sudden change detected!
Temperature changed from 46°C to 52°C
C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.exe
(process 30860) exited with code 0 (0x0).
Press any key to close this window . . .

```

```

Output
Show output from: Build
Build started at 17:24...
1>----- Build started: Project: stt-lab11, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab11 -> C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.dll
***** Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped *****
***** Build completed at 17:24 and took 02.365 seconds *****

```

Figure 18: Task 4 Q2 - code and output

Reasoning: the event fired only when the absolute temperature change was greater than 5. The changes 25→28 and 28→30 were ignored. The jump 30→46 (difference 16) triggered both the main message and the warning because the difference exceeded 10. The jump 46→52 (difference 6) triggered only the main message.

1.4.5 Task 5 - Output Reasoning (Level 2)

Q1. Final output:

```
[Start]{Ping}(Nested)[Start]{Pong}(Nested)[End][End]
```

```

1  using System;
2  class NotifyEventArgs : EventArgs
3  {
4      public string Message { get; }
5      public NotifyEventArgs(string msg) => Message = msg;
6  }
7  class Notifier
8  {
9      public event EventHandler<NotifyEventArgs> OnNotify;
10     public void Trigger(string msg)
11     {
12         Console.WriteLine("[Start]");
13         OnNotify?.Invoke(this, new NotifyEventArgs(msg));
14         Console.WriteLine("[End]");
15     }
16 }
17 class Program
18 {
19     static void Main()
20     {
21         Notifier n = new Notifier();
22         n.OnNotify += (s, e) =>
23         {
24             Console.WriteLine("{0} {1}", s, e.Message);
25         };
26         n.OnNotify += (s, e) =>
27         {
28             Console.WriteLine("(Nested)");
29             if (e.Message == "Ping")
30                 ((Notifier)s).Trigger("Pong");
31         };
32         n.Trigger("Ping");
33     }
34 }

```

Microsoft Visual Studio Debug Console Output:

```

[Start]{Ping}(Nested)[Start]{Pong}(Nested)[End][End]
C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.exe
(process 36112) exited with code 0 (0x0).
Press any key to close this window . . .

```

Output Window:

```

Build started at 18:13...
1>----- Build started: Project: stt-lab11, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab11 -> C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.dll
***** Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped *****
***** Build completed at 18:13 and took 02.150 seconds *****

```

Figure 19: Task 5 Q1 - code and output

Reasoning: calling `Trigger("Ping")` first printed `[Start]`. The first subscriber printed `{Ping}`, and the second printed `(Nested)` and then invoked `Trigger("Pong")` because the message was "Ping". The nested trigger printed another `[Start]`, then the two subscribers printed `{Pong}` and `(Nested)`. Since the message was now "Pong", there was no further recursion. Each `Trigger` call ended by printing `[End]`, so two `[End]` markers appeared at the end.

Q2. Final output:

```
[Check]{High}[Check][Done](Alert)[Done]
```

```

1  using System;
2  class AlertEventArgs : EventArgs
3  {
4      public string Info { get; }
5      public AlertEventArgs(string info) => Info = info;
6  }
7  class Sensor
8  {
9      public event EventHandler<AlertEventArgs> ThresholdReached;
10     public void Check(int value)
11     {
12         Console.WriteLine("[Check]");
13         if (value > 50)
14             ThresholdReached?.Invoke(this, new AlertEventArgs("High"));
15         Console.WriteLine("[Done]");
16     }
17 }
18 class Program
19 {
20     static void Main()
21     {
22         Sensor s = new Sensor();
23         s.ThresholdReached += (sender, e) =>
24         {
25             Console.WriteLine("{ " + e.Info + " }");
26             if (e.Info == "High")
27                 ((Sensor)sender).Check(30);
28         };
29         s.ThresholdReached += (sender, e) =>
30         {
31             Console.WriteLine("Alert");
32             s.Check(80);
33         }
34     }
35 }

```

Microsoft Visual Studio Debug Console Output:

```

[Check]{High}[Check][Done](Alert)[Done]
C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.exe (process 31516) exited with code 0 (0x0).
Press any key to close this window . . .

```

Output Window:

```

Show output from: Build
Build started at 19:10...
1>----- Build started: Project: stt-lab11, Configuration: Debug Any CPU -----
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>stt-lab11 -> C:\Users\shardul\source\repos\stt-lab11\stt-lab11\bin\Debug\net8.0\stt-lab11.dll
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 19:10 and took 02.211 seconds =====

```

Figure 20: Task 5 Q2 - code and output

Reasoning: `Check(80)` printed `[Check]` and then fired the threshold event because 80 was greater than 50. The first subscriber printed `{High}` and called `Check(30)` on the same sensor, which printed `[Check][Done]` but did not raise the event again because $30 \leq 50$. The second subscriber then printed `(Alert)`. After both subscribers finished, the original `Check(80)` call printed its final `[Done]`.

1.5 Conclusion

In this lab I built a Windows Forms example that used custom delegates and events to drive UI updates rather than direct control manipulation inside button handlers. I then extended the design using a custom `ColorEventArgs` and demonstrated multicast events by wiring both a label update and a message box to the same publisher. Overall the exercise strengthened my comfort with the publisher-subscriber pattern in C# and clarified practical details like passing `ref` parameters through multicast delegates, and composing `EventArgs` for extensibility without changing event signatures.

1.6 References

- Microsoft C# docs: <https://learn.microsoft.com/en-us/dotnet/csharp>
- Delegates and events: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/delegates-and-events>
- Windows Forms events: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/>